SYSTEM S4 — SOFTWARE DESIGN DOCUMENT

General Ledger, AR/AP, fiscal period management, and 24+ operational reports across all S-systems

CONSOLIDATES FORMER MODULES

M8 Finance & Accounting Service **M9 Reporting & Analytics Service**

DOCUMENT INFORMATION

Document type	Software Design Document — Developer Edition
Intended audience	The developer (or team) building System 4 — Finance & BI. Read S0 first.
Version	1.0
Date	2026-06-16
Companion documents	S0 Shared Platform Reference · S1 Identity & Access Platform · S2 Workforce & Payroll · S3 Hospitality Operations

Table of Contents

1. Purpose, Scope & Responsibilities

1.1 Why S4 exists

S4 is the central financial intelligence layer of Wonderland Hotel ERP. It maintains the entire double-entry general ledger, receives automated journal entries from S2 (Workforce & Payroll) and S3 (Hospitality Operations), manages Accounts Receivable and Accounts Payable sub-ledgers, closes fiscal periods aligned to the Ethiopian fiscal year (July–June), and presents 24+ operational reports and 5 executive dashboards to authorised users across all S-systems.

Every dirham of hotel revenue and every cost posted anywhere in the ERP eventually flows through S4 as a double-entry journal entry. S4 is the single source of truth for the general ledger and all derived financial statements.

1.2 Out of scope

CONVENTION

S4 does not create source business data. Employees, payroll runs, and leave records belong to S2. Reservations, orders, stock batches, and folios belong to S3. S4 receives financial summaries from those systems and posts them to the GL — it does not replicate or own source records.

Concern	Handled by
Employee and payroll records	S2 — Workforce & Payroll
Room reservations and folios	S3 — Hospitality Operations
Inventory and purchase orders	S3 — Hospitality Operations
Restaurant orders and bills	S3 — Hospitality Operations
Direct payment processing	S3 (folio/bill payments); S4 records the resulting GL entry only
User identity and JWT issuance	S1 — Identity & Access

1.3 Relationship to the original M8 / M9 services

Three simplifications result from consolidating M8 (Finance & Accounting) and M9 (Reporting & Analytics) into a single Laravel 11 application sharing one database (`wh_s4_db`).

DESIGN DECISION

ID	Decision	M8/M9 behaviour	S4 behaviour
IC1	M9 financial data access	M9 called M8 via `HTTP GET /api/v1/reports/*` for every dashboard load	S4 BI module queries `wh_s4_db` directly (in-process SQL) — no HTTP hop
IC2	BI cache invalidation after journal post	M8 fired `JournalPosted` Redis event; M9 `InvalidateReportCache` listener reacted	`JournalService::post()` calls `BiCacheService::invalidate()` in the same PHP process immediately

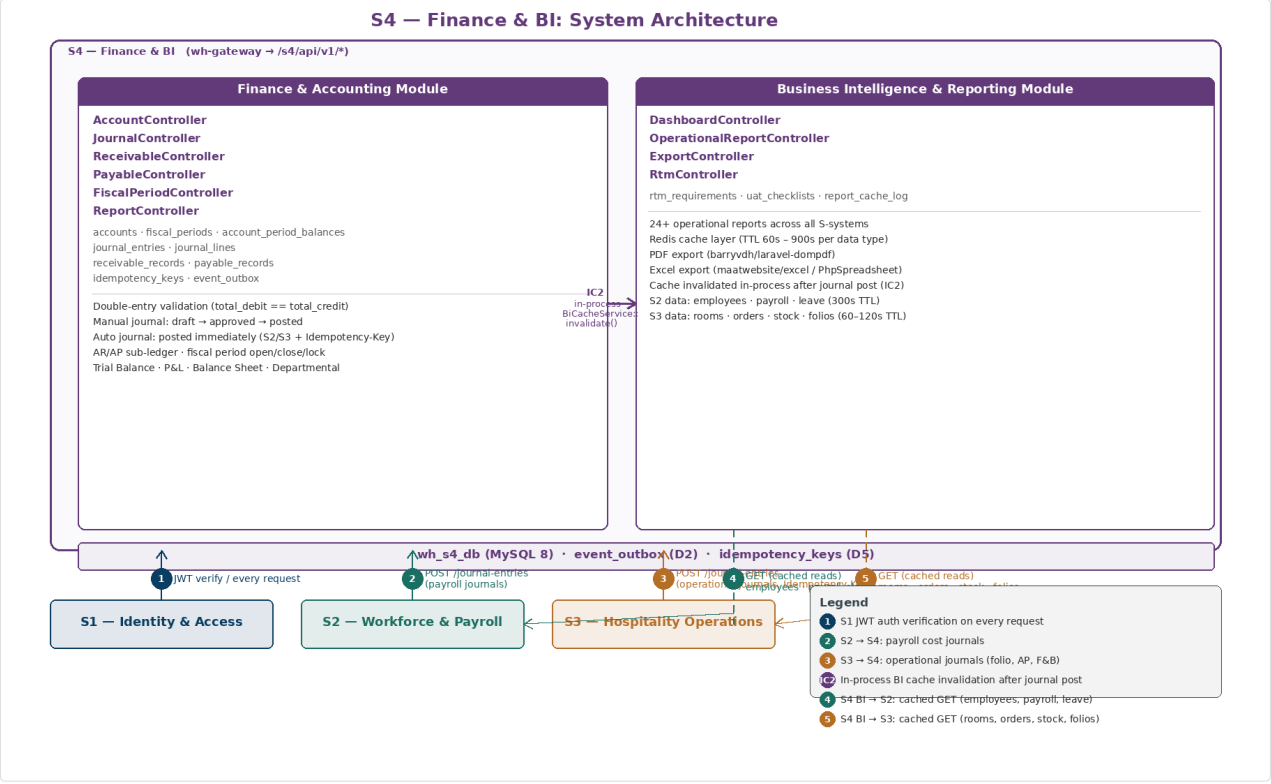
ID	Decision	M8/M9 behaviour	S4 behaviour
			after the DB transaction commits
IC3	RTM system codes	`rtm_requirements.module` used M-codes (`M1` – `M9`)	`rtm_requirements.s_code` uses S-codes (`S0` – `S4`) to reflect the consolidated architecture

2. Internal Architecture

2.1 Functional areas

S4 is one Laravel 11 application with two functional modules sharing `wh_s4_db`.

Module	Controllers	Core services	Responsibility
Finance & Accounting	AccountController, JournalController, ReceivableController, PayableController, FiscalPeriodController, ReportController	JournalService, ReceivableService, PayableService, ReportingService	GL journal posting, double-entry validation, AR/AP sub-ledgers, fiscal period close, four financial statements
BI & Reporting	DashboardController, OperationalReportController, ExportController, RtmController	DataAggregatorService, BiCacheService, PdfExportService, ExcelExportService	5 dashboards, 24+ reports from all S-systems, PDF/Excel export, RTM and UAT tracking



S4 system context diagram showing a single Laravel 11 application containing two module sub-boxes side by side. Finance & Accounting module (left) lists AccountController, JournalController, ReceivableController, PayableController, FiscalPeriodController, and ReportController; it notes in-process double-entry validation, fiscal period guard, AR/AP sub-ledger, and four financial statements. Business Intelligence & Reporting module (right) lists DashboardController, OperationalReportController, ExportController, and RtmController; it notes Redis cache TTLs of 60–900 s, PDF/Excel export, and RTM/UAT tracking. An IC2 in-process arrow labelled BiCacheService::invalidate() connects Finance to BI. A wh_s4_db cylinder sits below both modules. Badge 1 connects S1 for JWT auth; badge 2 connects S2 to S4 Finance for payroll journals; badge 3 connects S3 to S4 Finance for operational journals with Idempotency-Key; badge 4 (dashed) connects S4 BI to S2 for cached GET reads of employees, payroll, and leave; badge 5 (dashed) connects S4 BI to S3 for cached GET reads of rooms, orders, stock, and folios.

2.2 The IC1 / IC2 in-process simplifications

IC1 — eliminated M9 → M8 HTTP calls. In the original microservice design, every call to `GET /dashboard/finance` or `GET /reports/finance/*` in M9 triggered an internal HTTP request to M8's report endpoints. Under S4, both modules share the same database — `ReportingService::trialBalance($periodId)` runs a direct SQL aggregate query against `wh_s4_db.journal_lines` without any network round-trip.

IC2 — eliminated Redis event bus between Finance and BI. In M8+M9, after `JournalService::post()` persisted the journal, it fired a `JournalPosted` Redis event. M9 subscribed to that event via `InvalidateReportCache` and flushed the relevant cache keys. There was an inherent race window where a BI dashboard request could be served from stale cache between the journal commit and the listener firing. In S4, `JournalService::post()` calls `BiCacheService::invalidate(['finance.dashboard.*', "finance.period.{$fiscalPeriodId}.*"])` at the end of the same PHP request, before returning the 201 response. No Redis pub/sub subscription is required.

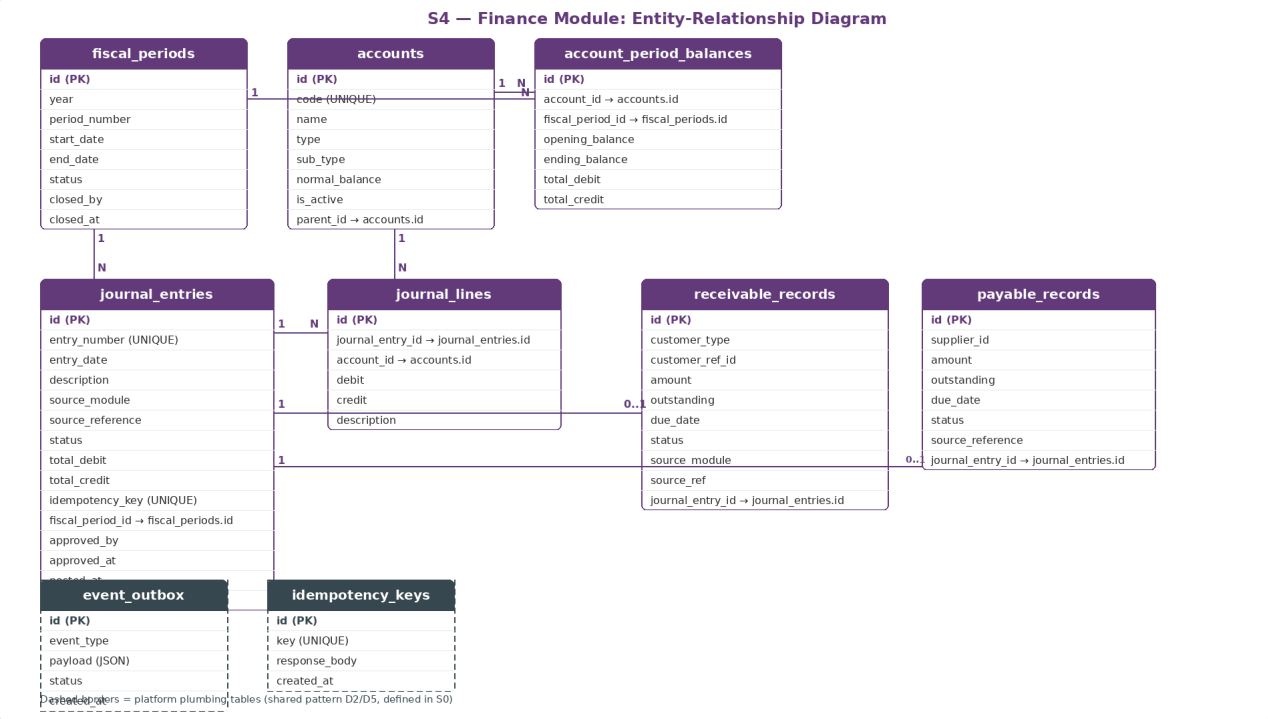
2.3 Folder structure

```
wh-s4-finance-bi/
├── app/
│   ├── Modules/
│   │   ├── Finance/
│   │   │   ├── Controllers/
│   │   │   │   ├── AccountController.php
│   │   │   │   ├── JournalController.php
│   │   │   │   ├── ReceivableController.php
│   │   │   │   ├── PayableController.php
│   │   │   │   ├── FiscalPeriodController.php
│   │   │   │   └── ReportController.php
│   │   │   └── Services/
│   │   │       ├── JournalService.php
│   │   │       ├── ReceivableService.php
│   │   │       ├── PayableService.php
│   │   │       └── ReportingService.php
│   │   └── Bi/
│   │       ├── Controllers/
│   │       │   ├── DashboardController.php
│   │       │   ├── OperationalReportController.php
│   │       │   ├── ExportController.php
│   │       │   └── RtmController.php
│   │       └── Services/
│   │           ├── DataAggregatorService.php
│   │           ├── BiCacheService.php
│   │           ├── PdfExportService.php
│   │           └── ExcelExportService.php
├── database/
│   └── migrations/   (12 files)
└── routes/
    └── api.php
```

3. Data Model

S4 owns eleven tables: seven Finance tables, two BI tables, and two platform plumbing tables (D2/D5). The two ER diagrams below show the relationships.

3.1 Finance & Accounting module



ER diagram for the S4 Finance module showing seven entities. fiscal_periods (id PK, year, period_number, start_date, end_date, status, closed_by, closed_at; UNIQUE year+period_number) sits in the top-left. accounts (id PK, code UNIQUE, name, type, sub_type, normal_balance, is_active, parent_id self-referencing) sits in the top-centre. account_period_balances (id PK, account_id FK, fiscal_period_id FK, opening_balance, ending_balance, total_debit, total_credit; UNIQUE account+period) sits top-right and has N:1 arrows to both fiscal_periods and accounts. journal_entries (id PK, entry_number UNIQUE, entry_date, description, source_module, source_reference, status, total_debit, total_credit, idempotency_key UNIQUE, fiscal_period_id FK, reversal_of_id FK, approved_by, posted_at, created_by) sits centre-left with a N:1 arrow to fiscal_periods. journal_lines (id PK, journal_entry_id FK, account_id FK, debit, credit, description) sits centre with 1:N arrows from journal_entries and N:1 to accounts. receivable_records (id PK, customer_type, customer_ref_id, amount, outstanding, due_date, status, source_module, source_ref, journal_entry_id FK) and payable_records (id PK, supplier_id, amount, outstanding, due_date, status, source_reference, journal_entry_id FK) sit centre-right with 0..1 arrows to journal_entries. Dashed event_outbox and idempotency_keys platform tables sit at the bottom.

Table: fiscal_periods

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
year	SMALLINT UNSIGNED	Gregorian year the Ethiopian period maps to (e.g. 2024)
period_number	TINYINT UNSIGNED	1 = July, 2 = August … 12 = June
start_date	DATE	First calendar day of the period
end_date	DATE	Last calendar day of the period

Column	Type / Constraint	Notes
status	ENUM(open, closing, closed, locked)	
closed_by	BIGINT UNSIGNED FK NULL	→ S2 employee reference
closed_at	TIMESTAMP NULL	
—	UNIQUE(year, period_number)	Prevents duplicate periods

Table: accounts

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
code	VARCHAR(20) UNIQUE	e.g. `1001`, `4002`
name	VARCHAR(150)	e.g. Cash and Cash Equivalents
type	ENUM(asset, liability, equity, income, expense)	
sub_type	VARCHAR(60) NULL	e.g. `current_asset`, `cost_of_sales`
normal_balance	ENUM(debit, credit)	DR for assets/expenses; CR for liabilities/equity/income
is_active	TINYINT(1) DEFAULT 1	Inactive accounts cannot receive new journal lines
parent_id	BIGINT UNSIGNED FK NULL	→ accounts.id — hierarchical chart of accounts

Table: journal_entries

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
entry_number	VARCHAR(20) UNIQUE	Auto-generated `JE-00001` sequence
entry_date	DATE	Must fall within an `open` or `closing` fiscal period
description	VARCHAR(255)	
source_module	ENUM(s2, s3, manual)	Originating system
source_reference	VARCHAR(100) NULL	e.g. `payroll_run_id:42`, `folio_id:71`
status	ENUM(draft, approved, posted, reversed)	Automated entries skip draft/approved
total_debit	DECIMAL(14,2)	Must equal `total_credit` (validated in JournalService)
total_credit	DECIMAL(14,2)	
idempotency_key	VARCHAR(100) UNIQUE NULL	Populated from `Idempotency-Key` header on automated postings
fiscal_period_id	BIGINT UNSIGNED FK	

Column	Type / Constraint	Notes
		→ fiscal_periods.id; derived from `entry_date`
reversal_of_id	BIGINT UNSIGNED FK NULL	→ journal_entries.id; set only on reversal entries
approved_by	BIGINT UNSIGNED NULL	S2 employee id of final approver
approved_at	TIMESTAMP NULL	
posted_at	TIMESTAMP NULL	
created_by	BIGINT UNSIGNED	S2 employee id or 0 for service-key automated postings

Table: journal_lines

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
journal_entry_id	BIGINT UNSIGNED FK	→ journal_entries.id
account_id	BIGINT UNSIGNED FK	→ accounts.id
debit	DECIMAL(12,2) DEFAULT 0.00	
credit	DECIMAL(12,2) DEFAULT 0.00	
description	VARCHAR(255) NULL	Line-level memo

Table: receivable_records

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
customer_type	ENUM(hotel_guest, event)	
customer_ref_id	BIGINT UNSIGNED	`guest_profile_id` or `event_id` from S3
amount	DECIMAL(12,2)	Original AR amount
outstanding	DECIMAL(12,2)	Remaining unpaid balance
due_date	DATE NULL	
status	ENUM(open, partial, settled, written_off)	
source_module	VARCHAR(5)	`s3`
source_ref	VARCHAR(100)	`folio_id:71` or `bill_id:112`
journal_entry_id	BIGINT UNSIGNED FK	→ journal_entries.id (originating GL entry)

Table: payable_records

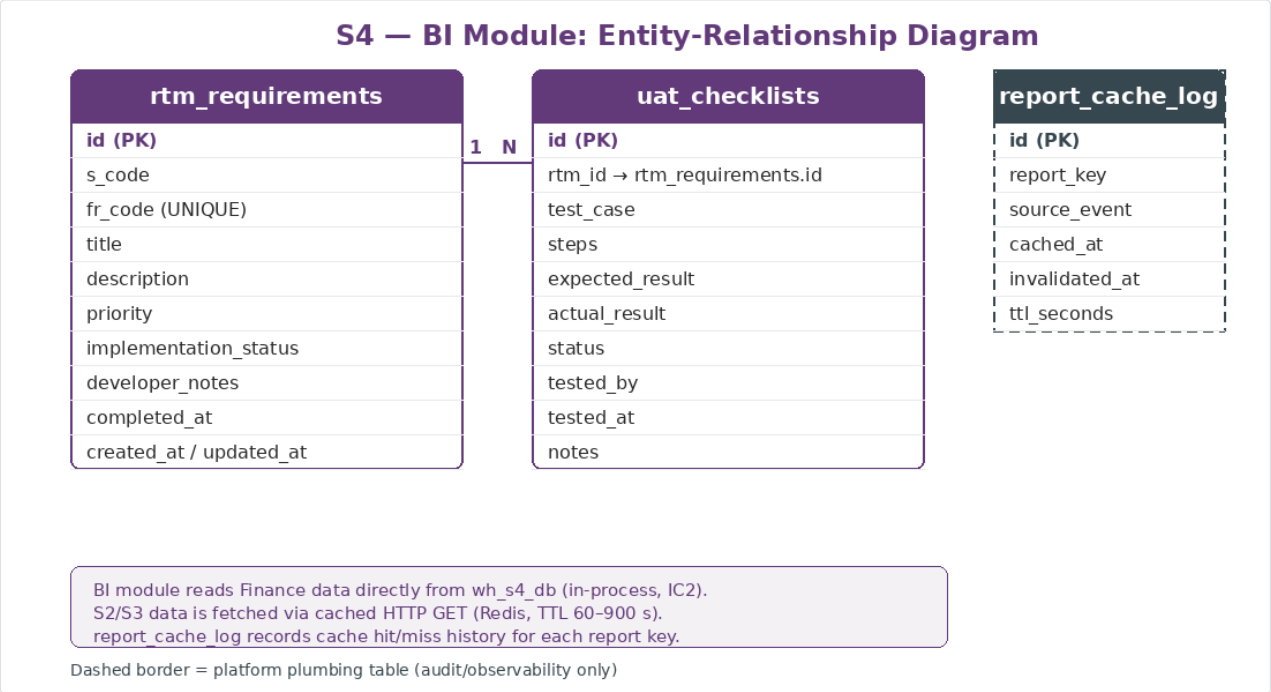
Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	

Column	Type / Constraint	Notes
supplier_id	BIGINT UNSIGNED	S3 supplier reference
amount	DECIMAL(12,2)	Original AP amount
outstanding	DECIMAL(12,2)	Remaining unpaid balance
due_date	DATE NULL	
status	ENUM(open, partial, settled)	
source_reference	VARCHAR(100)	e.g. `purchase_order_id:88` from S3
journal_entry_id	BIGINT UNSIGNED FK	→ journal_entries.id (originating GL entry)

Table: account_period_balances

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
account_id	BIGINT UNSIGNED FK	→ accounts.id
fiscal_period_id	BIGINT UNSIGNED FK	→ fiscal_periods.id
opening_balance	DECIMAL(12,2)	Carried forward from previous period
ending_balance	DECIMAL(12,2)	Computed and frozen when period transitions to `closed`
total_debit	DECIMAL(14,2)	Sum of all debit journal lines in this period
total_credit	DECIMAL(14,2)	Sum of all credit journal lines in this period
—	UNIQUE(account_id, fiscal_period_id)	

3.2 Business Intelligence & Reporting module



ER diagram for the S4 BI module showing three entities. rtm_requirements (id PK, s_code, fr_code UNIQUE, title, description, priority, implementation_status, developer_notes, completed_at, timestamps) sits on the left. uat_checklists (id PK, rtm_id FK, test_case, steps, expected_result, actual_result, status, tested_by, tested_at, notes) sits on the right with a N:1 arrow from rtm_requirements (1:N). A dashed report_cache_log (id PK, report_key, source_event, cached_at, invalidated_at, ttl_seconds) sits at the far right. A note box states that the BI module reads Finance data directly from wh_s4_db in-process (IC2); S2/S3 data is fetched via cached HTTP GET with Redis TTLs of 60–900 seconds; report_cache_log records cache history per report key.

Table: rtm_requirements

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
s_code	VARCHAR(5)	`S0` – `S4` — consolidated system reference
fr_code	VARCHAR(20) UNIQUE	e.g. `FR-S4-05`
title	VARCHAR(255)	
description	TEXT	Full requirement text from FRD
priority	ENUM(critical, high, medium, low)	
implementation_status	ENUM(not_started, in_progress, completed, deferred)	
developer_notes	TEXT NULL	
completed_at	DATE NULL	
created_at / updated_at	TIMESTAMPS	

Table: uat_checklists

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
rtm_id	BIGINT UNSIGNED FK	→ rtm_requirements.id
test_case	VARCHAR(255)	Brief description of what is being tested
steps	TEXT	Step-by-step test procedure
expected_result	TEXT	
actual_result	TEXT NULL	Filled in during UAT session
status	ENUM(pending, passed, failed, blocked)	
tested_by	BIGINT UNSIGNED NULL	S2 `employee_id` of UAT tester
tested_at	TIMESTAMP NULL	
notes	TEXT NULL	

Platform plumbing tables

Table	Pattern	Purpose
`event_outbox`	D2 (S0 §5.4)	Reliably publishes `wh.events.s4.*` events to the message bus
`idempotency_keys`	D5 (S0 §9)	Deduplicate repeated `POST /journal-entries` from S2/S3

4. API Endpoint Catalog

All routes are served under the prefix `/s4/api/v1/` via `wh-gateway`. Every request must carry a valid JWT (S1-issued) — except automated journal postings from S2/S3, which use `X-Service-Key` authentication without a user JWT.

4.1 Finance & Accounting module

Method	Endpoint	Description	Auth / Permission
GET	`/accounts`	List chart of accounts (hierarchical, filterable by `type`)	`S4.finance.accounts.read`
POST	`/accounts`	Create a new account	`S4.finance.accounts.create`
PUT	`/accounts/{id}`	Update name, sub_type, or is_active	`S4.finance.accounts.update`
POST	**`/journal-entries`**	**Post journal entry (automated or manual)**	**Service-Key + Idempotency-Key (auto) / JWT + `S4.finance.journal_entries.create` (manual)**
GET	`/journal-entries`	List entries (filter: `source_module`, `status`, `entry_date` range, `fiscal_period_id`)	`S4.finance.journal_entries.read`
GET	`/journal-entries/{id}`	Entry detail with all `journal_lines`	`S4.finance.journal_entries.read`
PUT	`/journal-entries/{id}/approve`	Approve manual `draft` or `approved` entry	`S4.finance.journal_entries.approve`
POST	`/journal-entries/{id}/reverse`	Reverse a `posted` entry (creates mirror entry)	`S4.finance.journal_entries.reverse`
GET	`/receivables`	AR list with aging buckets; filter by `customer_type`, `status`	`S4.finance.receivables.read`
POST	`/receivables/{id}/settle`	Record full or partial AR settlement	`S4.finance.receivables.settle`
GET	`/payables`	AP list with aging buckets; filter by `supplier_id`, `status`	`S4.finance.payables.read`
POST	`/payables/{id}/settle`	Record full or partial AP settlement	`S4.finance.payables.settle`
GET	`/fiscal-periods`	List all fiscal periods	`S4.finance.fiscal_periods.read`
POST	`/fiscal-periods`	Open the next fiscal period	`S4.finance.fiscal_periods.create`

Method	Endpoint	Description	Auth / Permission
PUT	`/fiscal-periods/{id}/close`	Initiate close (status → `closing` → `closed`)	`S4.finance.fiscal_periods.close`
PUT	`/fiscal-periods/{id}/lock`	Lock closed period post-audit (status → `locked`)	`S4.finance.fiscal_periods.lock`
GET	`/reports/trial-balance`	Trial balance aggregated by period	`S4.finance.reports.read`
GET	`/reports/profit-loss`	P&L statement (revenue – expenses)	`S4.finance.reports.read`
GET	`/reports/balance-sheet`	Balance sheet as of a given date	`S4.finance.reports.read`
GET	`/reports/departmental`	Revenue breakdown by department/account	`S4.finance.reports.read`

4.2 Business Intelligence & Reporting module

Method	Endpoint	Description	Auth / Permission
GET	`/dashboard/executive`	KPI dashboard — today's revenue, occupancy, payroll cost	`S4.bi.dashboards.read`
GET	`/dashboard/hotel`	Hotel occupancy and room revenue dashboard	`S4.bi.dashboards.read`
GET	`/dashboard/restaurant`	F&B daily sales dashboard	`S4.bi.dashboards.read`
GET	`/dashboard/finance`	Finance summary dashboard (AR, AP, period status)	`S4.bi.dashboards.read`
GET	`/reports/hr/employees`	Employee directory (filter: dept, education, status)	`S4.bi.reports.read`
GET	`/reports/hr/disciplinary`	Disciplinary history report	`S4.bi.reports.read`
GET	`/reports/payroll/summary`	Payroll summary sheet	`S4.bi.reports.read`
GET	`/reports/payroll/payslip/{emp_id}/{period}`	Individual payslip PDF	`S4.bi.reports.read`
GET	`/reports/payroll/pension`	Pension contribution report	`S4.bi.reports.read`
GET	`/reports/leave/balance`	Leave balance by employee	`S4.bi.reports.read`
GET	`/reports/inventory/stock-movement`	Stock movement history	`S4.bi.reports.read`
GET	`/reports/inventory/expiry`	Expiry alert report	`S4.bi.reports.read`
GET	`/reports/inventory/valuation`	Inventory valuation (FIFO/FEFO unit costs)	`S4.bi.reports.read`
GET	`/reports/restaurant/sales-by-type`	F&B sales by customer type	`S4.bi.reports.read`
GET	`/reports/hotel/occupancy`	Room occupancy by period	`S4.bi.reports.read`
GET	`/reports/hotel/outstanding-balances`	Outstanding folio balance tracker	`S4.bi.reports.read`
GET	`/reports/hotel/cashier-shift`	Cashier shift reconciliation report	`S4.bi.reports.read`
GET	`/reports/finance/ar-aging`	AR aging (0–30, 31–60, 61–90, 90+ days)	`S4.bi.reports.read`
GET	`/reports/finance/ap-aging`	AP aging (0–30, 31–60, 61–90, 90+ days)	`S4.bi.reports.read`

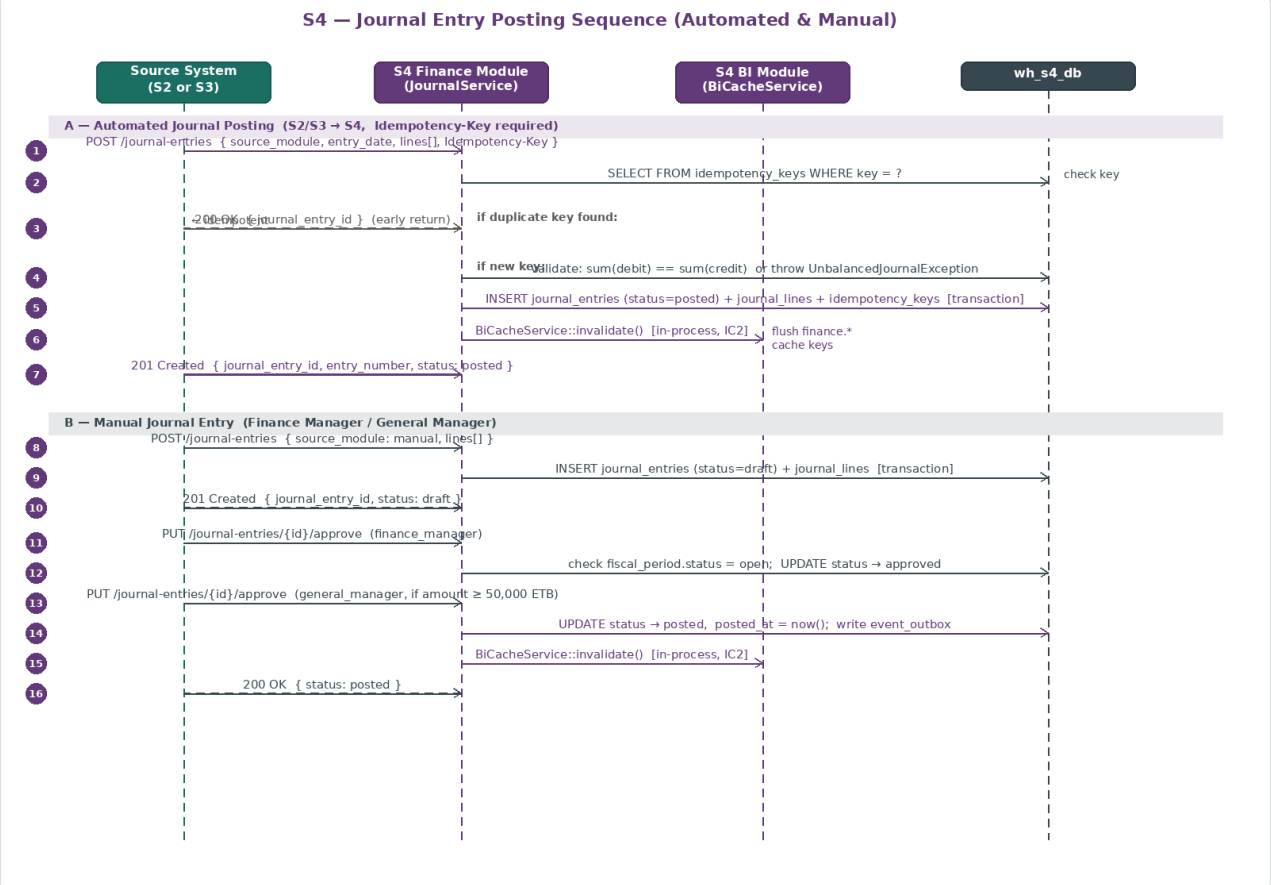
Method	Endpoint	Description	Auth / Permission
GET	`/reports/finance/profit-loss`	P&L from BI aggregation layer	`S4.bi.reports.read`
GET	`/reports/{any}?export=pdf`	Export any report as PDF (hotel letterhead)	`S4.bi.export.create`
GET	`/reports/{any}?export=excel`	Export any report as Excel workbook	`S4.bi.export.create`
GET	`/rtm`	Full RTM with implementation status per FR	`S4.bi.rtm.read`
PUT	`/rtm/{id}`	Update `implementation_status` or `developer_notes`	`S4.bi.rtm.update`
GET	`/rtm/{id}/uat`	UAT checklist for a requirement	`S4.bi.uat.read`
PUT	`/rtm/{id}/uat/{test_id}`	Record UAT test result	`S4.bi.uat.update`

> **NOTE**
>
> **Pagination, errors, rate limits.** All list endpoints use `?page=` / `?per_page=` cursor-free pagination (D6). Error responses follow the standard `{ "error": { "code", "message", "details" } }` envelope (D7). All endpoints are subject to the platform-wide rate limit defined in S0 §10 unless otherwise noted.

5. Core Business Logic

This section describes S4's key calculation engines and process flows in prose and tables. No PHP code is shown — the descriptions are precise enough for a developer to implement directly in `app/Modules/Finance/Services/*` and `app/Modules/Bi/Services/*`.

5.1 Journal posting sequence overview



Sequence diagram showing 16 numbered steps across four actors (Source System S2 or S3, S4 Finance Module JournalService, S4 BI Module BiCacheService, and wh_s4_db) grouped under two dividers. Divider A — Automated Journal Posting: Source System POSTs /journal-entries with source_module, entry_date, lines[], and Idempotency-Key header; S4 checks idempotency_keys table and returns 200 OK early if duplicate; if new, validates sum(debit)==sum(credit) or throws UnbalancedJournalException; inserts journal_entries (status=posted), journal_lines, and idempotency_keys in one DB transaction; calls BiCacheService::invalidate() in-process (IC2); returns 201 Created. Divider B — Manual Journal Entry: Finance Manager POSTs journal with source_module=manual; S4 inserts journal_entries (status=draft); Finance Manager PUTs /approve; S4 checks fiscal_period open and sets approved; General Manager approves if amount >= 50,000 ETB; S4 sets status=posted, calls BiCacheService::invalidate() in-process; returns 200 OK.

The numbered steps above correspond to the sequence diagram. Sections 5.2–5.8 expand the key process areas.

5.2 Double-entry validation and fiscal period guard

`JournalService::post($data)` enforces two hard rules before writing anything to the database:

Rule 1 — Balanced entry. `round(sum($lines[*].debit), 2)` must exactly equal `round(sum($lines[*].credit), 2)`. Any imbalance causes an `UnbalancedJournalException` (HTTP 422)

and no rows are written. This rule applies equally to automated postings from S2/S3 and to manually submitted entries.

Rule 2 — Open fiscal period. The `entry_date` on the incoming payload must fall within a `fiscal_periods` row whose `status` is `open` or `closing`. If the resolved period is `closed` or `locked`, the service throws a `ClosedPeriodException` (HTTP 422). S4 derives `fiscal_period_id` from `entry_date` using a simple lookup: `SELECT id FROM fiscal_periods WHERE start_date <= ? AND end_date >= ?`.

If both rules pass, `JournalService` opens a single database transaction and writes `journal_entries`, all `journal_lines`, and the `idempotency_keys` row atomically. The entry is assigned a sequenced `entry_number` (format `JE-{zero-padded-5-digit}`) before commit.

5.3 Automated journal sources

S4 receives automated journal entries from two upstream systems. Each call must include an `X-Service-Key` header (validated against `S4_SERVICE_KEY` in `.env`) and an `Idempotency-Key` header. The service key bypasses JWT authentication; the user-facing permission `S4.finance.journal_entries.create` is not required for automated postings.

Source	When posted	GL mapping
S2 — Payroll	After a payroll run is approved	Dr 5001 Salaries Expense / Cr 2100 Salaries Payable + 2101 Employee Pension + 2102 Employer Pension + 2200 Income Tax Payable
S3 — Goods receipt	After a purchase order receives goods	Dr 1200/1201 Inventory (per line, food vs. supplies) / Cr 2001 AP — Suppliers
S3 — Supplier payment	After a supplier payment is recorded	Dr 2001 AP — Suppliers / Cr 1001 Cash or 1002 Bank or 1003 Mobile Banking or 1004 POS (per payment method)
S3 — Folio settled	After a guest folio is settled	Dr 1001/1002/1003/1004/1005 (by payment method) / Cr 4001 Room Revenue + 4002 F&B Revenue + 4003 Service Charge Income + 2300 VAT Payable
S3 — Daily F&B summary	End-of-day batch (excludes `hotel_guest` orders, ID3)	Dr non-hotel-guest payment methods + 1101 AR Events / Cr 4002 F&B Revenue + 4003 Service Charge Income + 2300 VAT Payable

CONVENTION

Idempotency on automated postings. S2 and S3 must include `Idempotency-Key: {source_reference_unique_id}` on every `POST /journal-entries` call. `JournalService` checks `idempotency_keys` before processing: if the key exists, it returns `200 OK` with the existing `journal_entry_id` without inserting any new rows. If the key is absent, it processes normally and writes the key after a successful commit. This prevents duplicate GL entries on retry after network failure.

5.4 Manual journal approval workflow

Manual journal entries are created by `finance_manager` or `accountant` roles via the standard `POST /journal-entries` endpoint with `source_module: manual`. The two-tier approval workflow is enforced by `JournalController::approve()`.

Step	Actor	Action	`journal_entries.status`
1	`finance_manager` or `accountant`	`POST /journal-entries` with `source_module: manual`	`draft`
2	`finance_manager`	`PUT /journal-entries/{id}/approve`	`approved`
3a	Auto-post (if `total_debit` < 50,000 ETB)	Finance manager approval alone suffices	`posted`
3b	`general_manager` approval (if `total_debit` ≥ 50,000 ETB)	`PUT /journal-entries/{id}/approve`	`posted`

Once an entry reaches `posted`, `JournalService` sets `posted_at = now()`, records `approved_by`, and calls `BiCacheService::invalidate()` in-process (IC2). A manual draft entry may be deleted by its creator while in `draft` status; once `approved` or `posted`, deletion is forbidden — only reversal is permitted.

> **CONVENTION**
>
> **Fiscal period check on approval.** `JournalController::approve()` re-validates the fiscal period guard at approval time. If the period that contained `entry_date` has transitioned to `closing`, `closed`, or `locked` between draft creation and approval, the approval is rejected with `ClosedPeriodException`. The entry must be recreated with a date inside the current open period.

5.5 Journal reversal

`JournalController::reverse()` accepts `POST /journal-entries/{id}/reverse` on any entry with `status = posted`. The reversal:

1. Creates a new `journal_entries` row with `reversal_of_id = {original_id}`, `source_reference = "reversal_of:{entry_number}"`, and `status = posted` (reversals are auto-posted).
2. Copies every `journal_lines` row from the original, swapping `debit` and `credit` amounts (all debits become credits and vice versa).
3. The original entry's `status` remains `posted` — it is never mutated. The ledger balance is restored through the equal-and-opposite reversal entry.
4. The reversal entry is assigned to the current open fiscal period using `entry_date = today()`, not the original `entry_date`.

5.6 AR/AP sub-ledger management and aging buckets

`ReceivableService` and `PayableService` maintain sub-ledger records alongside each posting:

- When S3 posts a folio-settled journal that includes `Cr 1100 AR Hotel Guests` or `Cr 1101 AR Events`, `JournalService` also creates a `receivable_records` row linked to `journal_entry_id`.
- When S3 posts a goods-receipt journal that includes `Cr 2001 AP — Suppliers`, `JournalService` creates a `payable_records` row.

Aging buckets are computed dynamically at query time from `today() - due_date`:

Bucket	Days overdue
Current	0 days (not yet due)
0–30 days	1–30
31–60 days	31–60
61–90 days	61–90
90+ days	> 90

`POST /receivables/{id}/settle` and `POST /payables/{id}/settle` accept `{ "amount": decimal, "payment_method": string }`. The service decrements `outstanding` and transitions `status` from `open` → `partial` → `settled` as `outstanding` reaches zero. Each settlement also posts a corresponding GL journal entry (`Dr 1001/1002 / Cr 1100/1101` for AR; `Dr 2001 / Cr 1001/1002` for AP).

5.7 Fiscal period close workflow

Wonderland Hotel uses the Ethiopian fiscal year: **period 1 = July, period 12 = June**. S4 opens fiscal periods explicitly via `POST /fiscal-periods` (typically run by an Artisan command: `php artisan s4:fiscal-period:open`).

Stage	Status	Who transitions	What happens
Active month	`open`	(automatic after creation)	All journal entries accepted
Month-end reconciliation	`closing`	`finance_manager` via `PUT /fiscal-periods/{id}/close`	New entries still accepted; AR/AP aging and trial balance reviewed
Period confirmed	`closed`	`finance_manager` confirms all reconciling items resolved	No new journal entries accepted; `account_period_balances` rows computed and frozen
Post-audit lock	`locked`	`general_manager` via `PUT /fiscal-periods/{id}/lock`	Immutable — no entries, no changes, no reversals

When period transitions to `closed`, `ReportingService::closePeriod($period)` computes `account_period_balances` for every active account: sums all debit and credit journal lines for that period, derives `ending_balance`, and stores `ending_balance` as the `opening_balance` for the next period if it already exists.

> **CONVENTION**
>
> **D13 — period immutability.** Any `POST /journal-entries` or `POST /journal-entries/{id}/reverse` with an `entry_date` falling in a `closed` or `locked` period is rejected with HTTP 422 `ClosedPeriodException`. This rule applies regardless of `source_module` — even automated postings from S2/S3 are rejected. If S2/S3 need to correct an entry in a closed period, they must post a new correction entry in the current open period.

5.8 BI data aggregation and Redis cache strategy

`DataAggregatorService` serves as the single data-fetch layer for all dashboard and report controllers. It distinguishes between two data sources:

S4-local Finance data (IC1 — no HTTP). Calls to `DataAggregatorService::getFinance($key, $query, $ttl)` execute the provided `$query` callback directly against `wh_s4_db` (standard Eloquent query), then cache the result in Redis under `finance.{$key}` with the configured TTL. This eliminates the HTTP hop that existed in M9.

S2/S3 remote data (cached HTTP GET). Calls to `DataAggregatorService::getRemote($system, $path, $ttl)` issue `HTTP GET {system_url}{path}` with the `X-Service-Key` header and cache the result in Redis under `s{system}.{path_hash}` with the given TTL. On cache miss, the remote call is made and result cached. On remote service unavailability, the last cached value is returned with a `X-Cache: STALE` response header.

Data category	Source	Redis key prefix	Default TTL
Today's revenue / live F&B sales	S4 Finance DB (local)	`finance.revenue.today`	60 s

Data category	Source	Redis key prefix	Default TTL
Live occupancy rate	S3 (remote GET)	`s3.occupancy`	60 s
F&B daily sales summary	S3 (remote GET)	`s3.fnb.daily`	60 s
Inventory stock levels	S3 (remote GET)	`s3.stock`	120 s
Employee counts, leave balances	S2 (remote GET)	`s2.employees`, `s2.leave`	300 s
Payroll period summaries	S2 (remote GET)	`s2.payroll`	300 s
Financial reports (P&L, Balance Sheet)	S4 Finance DB (local)	`finance.reports.*`	900 s
RTM and UAT data	S4 BI DB (local)	`bi.rtm.*`	No cache — always fresh

Cache invalidation (IC2). After `JournalService::post()` completes, it calls `BiCacheService::invalidate(['finance.revenue.*', "finance.reports.*", "finance.period.{$period->id}.*"])`. This flushes all affected Redis keys synchronously in the same PHP request. No Redis event subscription is required.

5.9 Report export (PDF and Excel)

All report endpoints support an `?export=pdf` or `?export=excel` query parameter. `ExportController::export()` routes the request to `PdfExportService` (using `barryvdh/laravel-dompdf` with the Wonderland Hotel letterhead template) or `ExcelExportService` (using `maatwebsite/excel` / PhpSpreadsheet). Both outputs include: report title, applied filters, date range, `generated_at` timestamp, and `generated_by` name from the authenticated JWT.

6. Role-Based Access Control

6.1 Permission catalog

S4 permissions follow the platform naming convention `S4.{domain}.{resource}.{action}` (D12) where `domain` is `finance` or `bi`.

Permission	Description
`S4.finance.accounts.read`	View chart of accounts
`S4.finance.accounts.create`	Add new account codes
`S4.finance.accounts.update`	Edit account name, sub_type, is_active
`S4.finance.journal_entries.read`	View journal entries and lines
`S4.finance.journal_entries.create`	Submit manual journal entry
`S4.finance.journal_entries.approve`	Approve manual draft or approved entry
`S4.finance.journal_entries.reverse`	Reverse a posted entry
`S4.finance.receivables.read`	View AR sub-ledger and aging
`S4.finance.receivables.settle`	Record AR settlement
`S4.finance.payables.read`	View AP sub-ledger and aging
`S4.finance.payables.settle`	Record AP settlement
`S4.finance.fiscal_periods.read`	View fiscal period list and status
`S4.finance.fiscal_periods.create`	Open a new fiscal period
`S4.finance.fiscal_periods.close`	Initiate and confirm period close
`S4.finance.fiscal_periods.lock`	Lock a closed period post-audit
`S4.finance.reports.read`	Access trial balance, P&L, balance sheet, departmental
`S4.bi.dashboards.read`	Access any BI dashboard
`S4.bi.reports.read`	Access any operational report
`S4.bi.export.create`	Export reports to PDF or Excel
`S4.bi.rtm.read`	View RTM requirements
`S4.bi.rtm.update`	Update implementation status or developer notes
`S4.bi.uat.read`	View UAT checklist
`S4.bi.uat.update`	Record UAT test result

6.2 System roles — S4 access summary

Permission group	`super_admin`	`general_manager`	`finance_manager`	`accountant`	`dept_head`	Other staff
`S4.finance.accounts.*`	All	Read	All	Read	—	—
	✓	✓	✓	✓	—	—

Permission group	super_admin	general_manager	finance_manager	accountant	dept_head	Other staff
S4.finance.journal_entries.read						
S4.finance.journal_entries.create	✓	—	✓	✓	—	—
S4.finance.journal_entries.approve	✓	✓	✓	—	—	—
S4.finance.journal_entries.reverse	✓	✓	✓	—	—	—
S4.finance.receivables.* / payables.*	All	Read	All	Read	—	—
S4.finance.fiscal_periods.close	✓	✓	✓	—	—	—
S4.finance.fiscal_periods.lock	✓	✓	—	—	—	—
S4.finance.reports.read	✓	✓	✓	✓	—	—
S4.bi.dashboards.read	✓	✓	✓	✓	✓	—
S4.bi.reports.read	✓	✓	✓	✓	Own dept	—
S4.bi.export.create	✓	✓	✓	✓	✓	—
S4.bi.rtm.* / uat.*	All	All	All	All	Read	Read

6.3 CheckPermission middleware contract

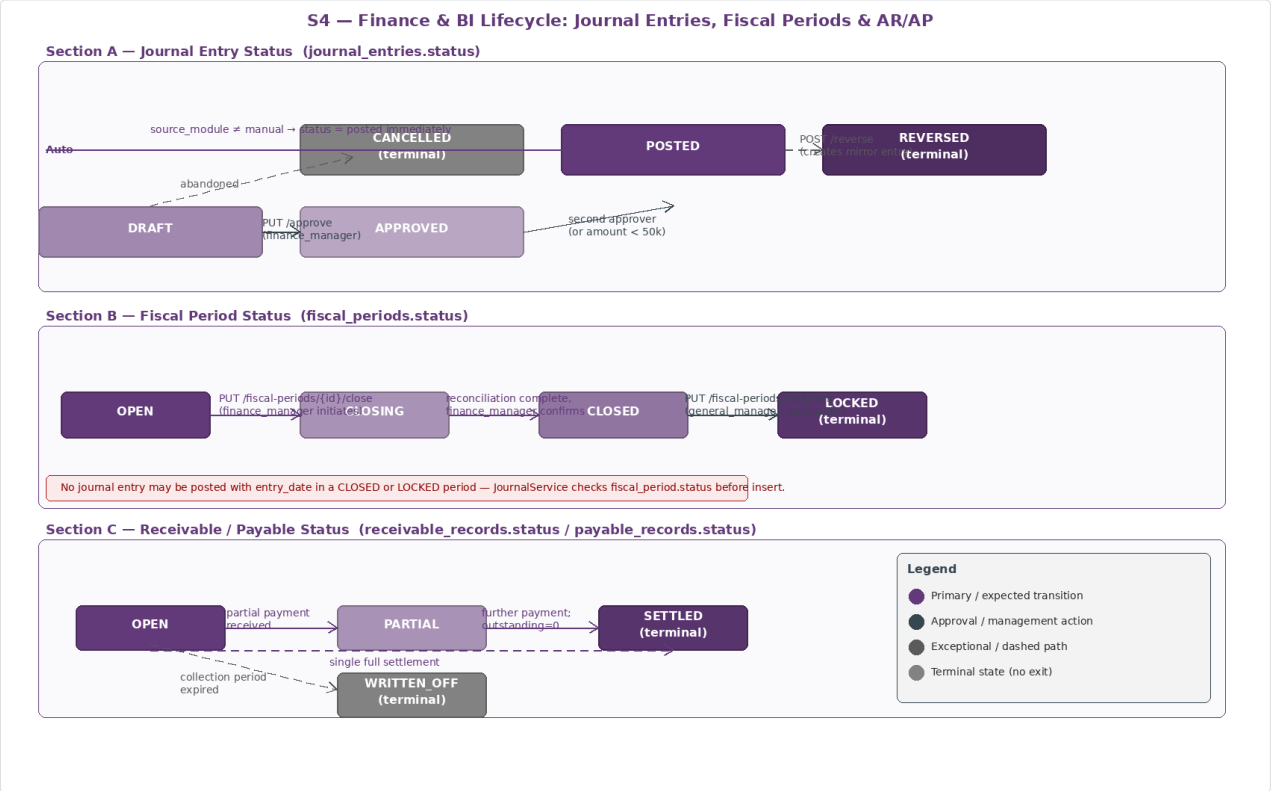
S4 applies `CheckPermission` middleware (D12) on all routes. For service-key authenticated calls (automated journal postings from S2/S3), the middleware short-circuits the permission check when a valid `X-Service-Key` header is present. All other routes require both a valid JWT and the route-specific permission.

6.4 Permission seeding

`php artisan db:seed --class=S4PermissionSeeder` creates all `S4.*` permissions in the S1-managed permissions store and assigns the default bundles listed in §6.2.

7. Finance & BI Lifecycle

7.1 Journal entry, fiscal period and AR/AP state machines



State diagram with three sections. Section A — Journal Entry Status: DRAFT state (manual entries only) transitions to APPROVED via PUT /approve (finance_manager); APPROVED transitions to POSTED (auto-post if total_debit < 50,000 ETB, or after general_manager second approval); automated entries (source_module != manual) skip directly to POSTED; POSTED transitions to REVERSED (terminal) via POST /reverse which creates a mirror entry; DRAFT may be CANCELLED (terminal) if abandoned. Section B — Fiscal Period Status: OPEN transitions to CLOSING (finance_manager initiates PUT /close); CLOSING transitions to CLOSED (finance_manager confirms all reconciling items resolved, account_period_balances computed); CLOSED transitions to LOCKED (terminal) via PUT /lock by general_manager post-audit; a rule box states no journal entry may be posted with entry_date in a CLOSED or LOCKED period. Section C — Receivable and Payable Status: OPEN transitions to PARTIAL (partial payment received); PARTIAL to SETTLED (terminal, outstanding reaches zero); OPEN may go directly to SETTLED (single full settlement, dashed); OPEN to WRITTEN_OFF (terminal, dashed, after collection period expires).

Journal entry states

State	Meaning	Entered via
`draft`	Manual entry submitted; not yet approved	`POST /journal-entries` with `source_module: manual`
`approved`	First approver confirmed; awaiting second for ≥ 50,000 ETB entries	`PUT /journal-entries/{id}/approve` by `finance_manager`
`posted`	Fully approved and included in ledger balances	Auto-approval (automated source, or amount < 50k); or second approval for ≥ 50k
`reversed`	A reversal entry exists; original remains readable	Mirror-entry created via `POST /journal-entries/{id}/reverse`

Fiscal period states

State	Meaning	Entered via
`open`	Normal operating state; all postings accepted	Created by `POST /fiscal-periods`
`closing`	Month-end reconciliation phase; postings still accepted	`PUT /fiscal-periods/{id}/close` (first step, finance_manager)
`closed`	All reconciling items resolved; period balances frozen	Confirmed by `finance_manager`; `account_period_balances` computed
`locked`	Post-audit immutability; no entries, no reversals	`PUT /fiscal-periods/{id}/lock` by `general_manager`

AR/AP states

State	Meaning	Entered via
`open`	No payments received against this record	Created by `JournalService` on originating posting
`partial`	At least one payment received; `outstanding > 0`	`POST /receivables/{id}/settle` or `POST /payables/{id}/settle` with `amount < outstanding`
`settled`	`outstanding = 0` — terminal	Further payment reduces outstanding to zero
`written_off`	Outstanding balance written off after collection period	Manual action by `finance_manager`

8. Event-Driven Integration

8.1 Events emitted by S4

Event	Trigger	Key payload fields
`wh.events.s4.journal.posted`	Every `journal_entries` row transitions to `status = posted`	`journal_entry_id`, `entry_number`, `entry_date`, `source_module`, `source_reference`, `total_debit`, `fiscal_period_id`
`wh.events.s4.period.closed`	`fiscal_periods.status` transitions to `closed`	`fiscal_period_id`, `year`, `period_number`, `closed_by`, `closed_at`

Both events are written to `event_outbox` and published by the outbox relay (D2, S0 §5.4).

8.2 Events consumed by S4

S4 does **not** subscribe to events from any other S-system. Instead:

- S2 and S3 call `POST /s4/api/v1/journal-entries` synchronously after their own business events complete.
- BI cache invalidation is handled in-process (IC2) after each journal post — no Redis event subscription is required.

8.3 Synchronous calls received by S4

Caller	Endpoint	Frequency	Auth
S2 — Workforce & Payroll	`POST /s4/api/v1/journal-entries`	Once per approved payroll run	`X-Service-Key` + `Idempotency-Key`
S3 — Hospitality Operations	`POST /s4/api/v1/journal-entries`	Per folio settlement, per goods receipt, per AP payment, daily F&B summary	`X-Service-Key` + `Idempotency-Key`

8.4 Synchronous calls made by S4

Target	Endpoint	Frequency	Purpose
S1 — Identity & Access	`POST /s1/api/v1/auth/verify`	Every inbound request	JWT verification (D8)
S2 — Workforce & Payroll	`GET /s2/api/v1/*`	On BI cache miss	Employee, payroll, leave data for reports
S3 — Hospitality Operations	`GET /s3/api/v1/*`	On BI cache miss	Rooms, reservations, orders, stock, folios for reports

CROSS-REFERENCE

S4 outbox mechanics follow D2 (S0 §5.4). The `event_outbox` table is polled every 10 seconds by `EventRelayJob`. On successful publish to the message bus, `status` is set to `published`. On three consecutive failures, `status` is set to `failed` and an alert is raised to `super_admin`.

Idempotency mechanics follow D5 (S0 §9). `JournalService` checks `idempotency_keys.key = {Idempotency-Key header value}` before processing. If found, returns the cached response. If not found, processes and inserts the key post-commit. TTL for idempotency keys is 24 hours for automated postings.

9. Financial Reports & Chart of Accounts

Section 9 documents the standard chart of accounts seeded into `wh_s4_db` on first deployment, and the complete 24+ operational reports reference served by the S4 BI module. The financial statements produced directly by S4 — Trial Balance, P&L, Balance Sheet, and Departmental Revenue — are listed in §9.3.

9.1 Standard chart of accounts

The following account codes are seeded on first deployment via `AccountSeeder`. Custom accounts may be added via `POST /accounts` but must not conflict with these codes.

Code	Account Name	Type	Normal Balance
1001	Cash and Cash Equivalents	Asset	Debit
1002	Bank Account — CBE	Asset	Debit
1003	Mobile Banking Clearing Account	Asset	Debit
1004	POS Clearing Account	Asset	Debit
1005	VISA Clearing Account	Asset	Debit
1100	Accounts Receivable — Hotel Guests	Asset	Debit
1101	Accounts Receivable — Events	Asset	Debit
1200	Inventory — Food and Beverage	Asset	Debit
1201	Inventory — Supplies and Consumables	Asset	Debit
2001	Accounts Payable — Suppliers	Liability	Credit
2100	Salaries Payable	Liability	Credit
2101	Employee Pension Payable (7%)	Liability	Credit
2102	Employer Pension Payable (11%)	Liability	Credit
2200	Income Tax Withheld Payable	Liability	Credit
2300	VAT Payable	Liability	Credit
4001	Room Revenue	Income	Credit
4002	Food and Beverage Revenue	Income	Credit
4003	Service Charge Income	Income	Credit
4004	Event Revenue	Income	Credit
5001	Salaries and Wages Expense	Expense	Debit
5002	Pension Contribution Expense (Employer)	Expense	Debit
5003	Cost of Food Sold	Expense	Debit
5004	Supplies and Consumables Expense	Expense	Debit

9.2 Complete operational reports reference (24+)

Report Name	Source	Key Columns	Filters
Employee Directory	S2	Emp#, Name, Dept, Position, Education, Status	Dept, Education, Status
Disciplinary History	S2	Emp#, Date, Warning Type, Description, Status	Employee, Type, Date
Asset Clearance	S2	Emp#, Asset, Code, Issue Date, Return Date, Cleared?	Employee, Dept
Guarantor Letter (PDF)	S2	Employee, Guarantor, Employer, Signature lines	Employee ID
Payroll Summary Sheet	S2	Dept, Employee, Gross, Deductions, Net, Tax, Pension	Period, Dept
Individual Payslip PDF	S2	All earnings, all deductions, net pay	Employee, Period
Pension Contribution	S2	Employee, Basic, Emp 7%, Empr 11%, Total	Period
Overtime Report	S2	Employee, Date, Type, Hours, Multiplier, Amount	Period, Type, Dept
Leave Balance	S2	Employee, Leave Type, Accrued, Used, Pending, Available	Employee, Dept
Leave Utilisation	S2	Dept, Leave Type, Total Days Taken, Employees	Period, Dept
Stock Movement	S3	Item, Batch, Type, In/Out Qty, Date, Reference, Balance	Item, Category, Date
Expiry Alert	S3	Item, Batch, Qty Remaining, Expiry Date, Days Left	Days Threshold
Inventory Valuation	S3	Item, Strategy, Qty, Unit Cost, Total Value	Category
Supplier Payment History	S3	Supplier, Date, Amount, Method, Reference, Posted?	Supplier, Date
F&B Sales by Customer Type	S3	Date, Type, Orders, Subtotal, SC, VAT, Total	Period, Type
Employee Consumption	S3	Employee, Period, Items, Total Amount, Deducted?	Period, Employee
Event F&B Billing	S3	Event ID, Items, Subtotal, SC, VAT, Total	Event, Period
Room Occupancy	S3		Period

Report Name	Source	Key Columns	Filters
		Room, Type, Nights Occupied, Occupancy %, Revenue	
Guest Folio / Invoice	S3	All 13 mandatory invoice fields	Guest, Date Range
Outstanding Balances	S3	Guest, Folio, Total Charges, Paid, Balance, Days Overdue	Status
Cashier Shift Report	S3	Cashier, Shift, Cash, POS, VISA, Mobile, Bank, Grand Total	Cashier, Date
AR Aging	S4 Finance	Customer, Invoice, 0–30, 31–60, 61–90, 90+ days	Customer Type
AP Aging	S4 Finance	Supplier, Invoice, 0–30, 31–60, 61–90, 90+ days	Supplier
Profit & Loss	S4 Finance	Revenue, COGS, Gross Profit, Operating Expenses, Net Income	Period, Dept

9.3 Financial statements produced by S4

Statement	Source data	Available from
Trial Balance	Aggregate debit/credit per account per period from `journal_lines`	`GET /reports/trial-balance?period_id={id}`
Profit & Loss	Sum of income accounts minus sum of expense accounts for the period	`GET /reports/profit-loss?from={date}&to={date}`
Balance Sheet	Sum of asset, liability, and equity accounts as of a date	`GET /reports/balance-sheet?as_of={date}`
Departmental Revenue	Sum of revenue accounts (4001–4004) grouped by department/source	`GET /reports/departmental?period_id={id}`

10. Security, Secrets & Operational Conventions

10.1 Platform conventions (D1–D14) as applied to S4

Convention	S4 application
D1 — URL structure	All routes under `/s4/api/v1/`
D2 — Event outbox	`event_outbox` in `wh_s4_db` ; 2 event types
D3 — JWT auth (S1)	Required on all user-facing routes; service-key bypasses for S2/S3 automated postings
D5 — Idempotency	`idempotency_keys` table; 24-hour TTL for automated journal postings
D6 — Pagination	`?page=` / `?per_page=` on all list endpoints
D7 — Error envelope	`{ "error": { "code", "message", "details" } }` on all 4xx/5xx
D8 — Service-to-service auth	`X-Service-Key` on automated `POST /journal-entries` from S2/S3; `X-Service-Key` on outbound BI data pulls to S2/S3
D12 — Permission naming	`S4.{domain}.{resource}.{action}` — 23 permissions total
D13 — Immutability	Posted journal entries immutable; closed/locked periods block new entries

10.2 Environment configuration (`.env`)

```
APP_NAME=WH-Finance-BI-Service
APP_URL=http://s4-service:8004
APP_KEY=base64:<generated>

# Database
DB_HOST=wh-s4-db
DB_PORT=3306
DB_DATABASE=wh_s4_db
DB_USERNAME=wh_s4_user
DB_PASSWORD=<secret>
DB_ENCRYPT=true         # TLS connection to MySQL (financial data at rest)

# Redis
REDIS_HOST=wh-redis
REDIS_PORT=6379
CACHE_DRIVER=redis
QUEUE_CONNECTION=redis

# S1 — Identity & Access
S1_SERVICE_URL=http://s1-service:8001
S1_SERVICE_KEY=<shared-secret>

# S2 — Workforce & Payroll (BI data pulls)
S2_SERVICE_URL=http://s2-service:8002
S2_SERVICE_KEY=<shared-secret>

# S3 — Hospitality Operations (BI data pulls)
```

```
S3_SERVICE_URL=http://s3-service:8003
S3_SERVICE_KEY=<shared-secret>

# Fiscal year
FISCAL_YEAR_START_MONTH=7       # Ethiopian fiscal year starts July

# Cache TTLs (seconds)
CACHE_TTL_REVENUE=60
CACHE_TTL_OCCUPANCY=60
CACHE_TTL_PAYROLL=300
CACHE_TTL_STOCK=120
CACHE_TTL_EMPLOYEES=300
CACHE_TTL_FINANCIAL_REPORTS=900
REPORT_CACHE_DEFAULT_TTL=300

# Export
PDF_LETTERHEAD_PATH=storage/letterhead/wonderland_hotel.pdf

# Backup
BACKUP_SCHEDULE=daily
BACKUP_RETENTION_DAYS=90
```

11. Non-Functional Requirements & Testing

11.1 Performance targets

Metric	Target
Dashboard endpoint (Redis cache hit)	P95 ≤ 150 ms
Report generation (cache hit)	P95 ≤ 150 ms
Report generation (cache miss, local Finance data)	P95 ≤ 800 ms
Report generation (cache miss, remote S2/S3 data)	P95 ≤ 1,500 ms
`POST /journal-entries` (automated, idempotency check + DB insert)	P95 ≤ 400 ms
PDF / Excel export	P95 ≤ 3,000 ms
Concurrent report users	≥ 30 simultaneous, sustained

11.2 Security

- All financial data encrypted in transit (TLS) to MySQL (`DB_ENCRYPT=true`) and to Redis.
- `journal_entries` and `journal_lines` are read-only once `status = posted`; no UPDATE is permitted except to `reversed` by the reversal endpoint which creates a new row.
- `fiscal_periods.status = locked` is irreversible: no code path exists to unlock a period.
- All `S4.finance.*` actions are logged to the application audit log (timestamp, actor, action, resource_id).
- Automated service-key callers (S2, S3) are restricted to `POST /journal-entries` only; they cannot read financial reports or approve entries.

11.3 Observability

- `report_cache_log` records every cache hit and invalidation with `source_event`, enabling audit of which journal triggered which cache flush.
- Artisan command `php artisan s4:fiscal-period:open` — creates the next Ethiopian fiscal period row.
- Artisan command `php artisan s4:report-cache:purge` — manually flushes all `finance.*` and `s2.*` / `s3.*` cache keys (emergency use).
- Health check: `GET /s4/health` returns `200 OK` with DB ping and Redis ping status.

11.4 Testing requirements

Category	Requirement
Unit — double-entry validation	`JournalService::post()` must reject any imbalanced entry and any entry with a `source_module=s2/s3` missing an `Idempotency-Key`
Unit — period guard	`JournalService::post()` must reject entries with `entry_date` in `closed` or `locked` periods
Integration — idempotency	Identical `Idempotency-Key` submitted twice must produce exactly one `journal_entries` row

Category	Requirement
Integration — in-process cache invalidation	Posting a journal must flush `finance.dashboard.*` cache keys before returning 201
Integration — period close	After `PUT /fiscal-periods/{id}/close` (confirmed), `account_period_balances` rows must exist for every active account
UAT	Tracked in `uat_checklists` linked to `rtm_requirements` ; all critical FRs must be `passed` before go-live

12. Integration Contract Summary

12.1 What S4 provides

Consumer	What is provided	Endpoint / mechanism
S2 — Workforce & Payroll	Journal entry sink for payroll cost journals	`POST /s4/api/v1/journal-entries` (service-key)
S3 — Hospitality Operations	Journal entry sink for folio, inventory, F&B, and AP journals; cached financial report reads	`POST /s4/api/v1/journal-entries` (service-key); `GET /s4/api/v1/reports/*` (JWT)
All S-system users	BI dashboards and 24+ operational reports	`GET /s4/api/v1/dashboard/*` and `GET /s4/api/v1/reports/*` (JWT + S4.bi.* permissions)
Finance team	GL management, AR/AP sub-ledger, fiscal period close	Full Finance module API (JWT + S4.finance.* permissions)
Developers / QA	RTM requirement tracking and UAT sign-off	`GET /PUT /s4/api/v1/rtm/*` (JWT + S4.bi.rtm/uat permissions)

12.2 What S4 depends on

Dependency	Required for	Degrades to
S1 — Identity & Access	JWT verification on all user-facing requests	401 on every request if S1 is unreachable
S2 — Workforce & Payroll	BI report data (employees, payroll, leave)	Stale cached data returned with `X-Cache: STALE` header
S3 — Hospitality Operations	BI report data (rooms, orders, stock, folios)	Stale cached data returned with `X-Cache: STALE` header
Redis	BI cache layer; queue for event relay	Cache misses trigger live DB/HTTP queries; performance degrades but correctness is maintained
`wh_s4_db`	All Finance and BI local data	Service unavailable

12.3 Build-order implication

Because S1 issues every JWT that S4 validates, **S1 must be running before S4 can accept any user request**. S2 and S3 must be running for the BI dashboards to serve fresh data, but stale-cache fallback means partial dashboard availability while S2/S3 are being deployed. The recommended build order within S4 itself:

1. **Finance module first** — seed `accounts` chart, open the first fiscal period; automated journal postings from S2/S3 can begin immediately.
2. **BI module second** — depends on Finance data already being present in `wh_s4_db`; the first cache warm-up populates all Finance-sourced dashboard values.
3. **RTM seeding third** — `php artisan db:seed --class=RtmSeeder` loads all `S0`–`S4` functional requirements so UAT tracking is ready for the acceptance phase.

The S4 → S3 journal-entry contract (S0 §5.3 sync call IC3) should be agreed with the S3 developer before S3's folio settlement code is merged — specifically the exact `lines[]` payload structure and the `Idempotency-Key` format (`folio-{id}`).

End of Document — S4, Finance & BI, v1.0